

DEEP LEARNING

Report 1: Gradient Descent

Student: Vu Ha Vy – 2540112

Implementation

The gradient descent function is implemented using a `while` loop with a convergence threshold:

```
def gradient_descent(x0, lr, max_iter=1000, threshold=0.01):
    x = x0
    t = 1
    while t <= max_iter:
        fx = f(x)
        print(f"Time = {t} | x = {x:.4f} | f(x) = {fx:.4f}")
        x_new = x - lr * df(x)
        fx_new = f(x_new)
        if abs(fx_new - fx) < threshold:
            print(f"\n Converged after {t} iterations, f(x) = {fx_new:.4f}")
            break
        x = x_new
        t += 1
    else:
        print(f"\n Reached max iterations ({max_iter}), f(x) = {f(x):.4f}")
```

The experiment uses $x_0 = 10$, `max_iter` = 1000, `threshold` = 0.01, with four different learning rates for the function $f(x) = x^2$ ($df(x) = 2x$).

Analyze of Different Learning Rates

Large learning rate ($lr = 1.0$)

With $lr = 1$, x flips sign every step between 10 and -10. Since $f(x) = x^2$ gives the same value for $+x$ and $-x$, the change in $f(x)$ is exactly zero and the threshold is met after just 1 step, but at $f(x) = 100$ is definitely not the minimum value.

```
Time = 1 | x = 10.0000 | f(x) = 100.0000
```

```
Converged after 1 iterations, f(x) = 100.0000
```

Good learning rate ($lr = 0.1$)

The algorithm descends quickly and converges in 20 steps, reaching $f(x) \approx 0.013$, very close to the true minimum of 0, and match the table in the lecture.

```
Time = 1 | x = 10.0000 | f(x) = 100.0000
Time = 2 | x = 8.0000 | f(x) = 64.0000
Time = 3 | x = 6.4000 | f(x) = 40.9600
...
Time = 18 | x = 0.2252 | f(x) = 0.0507
Time = 19 | x = 0.1801 | f(x) = 0.0325
Time = 20 | x = 0.1441 | f(x) = 0.0208
```

```
Converged after 20 iterations, f(x) = 0.0133
```

Small learning rate ($lr = 0.01$)

The progress is steady but much slower. After 150 steps the threshold is met, but the final value is still $f(x) \approx 0.23$, farther from 0 than the $lr = 0.1$ case.

```

Time = 1   | x = 10.0000 | f(x) = 100.0000
Time = 2   | x =  9.8000 | f(x) =  96.0400
Time = 3   | x =  9.6040 | f(x) =  92.2368
...
Time = 148 | x =  0.5131 | f(x) =   0.2633
Time = 149 | x =  0.5029 | f(x) =   0.2529
Time = 150 | x =  0.4928 | f(x) =   0.2429

```

Converged after 150 iterations, $f(x) = 0.2333$

Very small learning rate ($lr = 0.001$)

The steps are so small that it takes 923 iterations to converge, with $f(x) \approx 2.48$. The result is far from the actual minimum of 0, so this learning rate is too small to be useful.

```

Time = 1   | x = 10.0000 | f(x) = 100.0000
Time = 2   | x =  9.9800 | f(x) =  99.6004
Time = 3   | x =  9.9600 | f(x) =  99.2024
...
Time = 921 | x =  1.5853 | f(x) =   2.5130
Time = 922 | x =  1.5821 | f(x) =   2.5030
Time = 923 | x =  1.5789 | f(x) =   2.4930

```

Converged after 923 iterations, $f(x) = 2.4830$

Comparison

Finally, I create a table of the effect of different learning rates on the gradient descent's performance.

Learning Rate	Iterations	Final $f(x)$	Result
1.0	1	100.0000	Diverged
0.1	20	0.0133	Converged
0.01	150	0.2333	Converged
0.001	923	2.4830	Converged

Table 1: Comparison of gradient descent with different learning rates.